MEKELLE UNIVERSITY
MEKELLE INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

GE'EZ SCRIPT RECOGNITION USING CONVOLUTIONAL RECURRENT NEURAL NETWORK WITH PREPROCESSING FOR HISTORICAL MANUSCRIPT DIGITIZATION

A Thesis Documentation Submitted to Mekelle Institute of Technology in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science and Engineering

Leake Adisalem[1*], Leake Abrha[1*], Freweyni Asgedom[1*], Biniyam Gebreyesus[1*]

[1]Department of Computer Science and Engineering, Mekelle Institute of Technology, Mekelle University, Mekelle, Ethiopia

*Co-first authors — contributed equally to this work

Advisor: Maarig Aregawi Hagos (Assistant Professor)

May 2026

Mekelle, Ethiopia

ABSTRACT

This thesis presents an automated recognition system for the Ge'ez script — an abugida used in Ethiopia and Eritrea — based on a hybrid Convolutional Recurrent Neural Network (CRNN) with Connectionist Temporal Classification (CTC) loss. The script's large syllabary (317 characters, plus a blank token), visually similar characters distinguished only by subtle diacritics, and degradation in historical manuscripts pose significant challenges. A tailored preprocessing pipeline handles border removal, skew correction, noise filtering, contrast enhancement, and binarization. The CRNN combines a five-block CNN feature extractor with a two-layer bidirectional LSTM and a fully connected output layer. Training on 57,891 clean line images was augmented threefold (≈121k samples/epoch). The final model achieves a test Character Error Rate of 0.58% (99.42% character accuracy). A complementary full-page extraction module applied to a sample page of 28 lines yields a page-level CER of 26.35%. These results demonstrate the viability of deep learning for large-syllabary abugida scripts, while the stark line-to-page performance gap underscores the need for more robust page-level layout and degradation handling.

ACKNOWLEDGMENTS

We would like to start by expressing our deepest gratitude to our advisor, Maarig Aregawi, whose guidance has been valuable throughout this research. His expertise in Machine Learning and Error Analysis helped shape this work in ways we could not have managed on our own. Having someone who understands both the technical challenges and the cultural importance of this project made a real difference.

We also want to thank the faculty and staff of the Computer Science and Engineering Department for providing a good research environment.

To our fellow students: thank you for the discussions and for keeping us grounded when things got difficult. And to our families — your support and encouragement carried us through every challenge. This thesis is as much yours as it is ours.

TABLE OF CONTENTS

Abstract i

Acknowledgments ii

Table of Contents iii

List of Figures v

List of Tables v

Acronyms and Abbreviations vi

CHAPTERS

1. INTRODUCTION 1

    1.1 Overview 1

    1.2 Problem Statement 2

    1.3 Research Questions 3

    1.4 Objectives 3

    1.5 Scope and Limitations 4

    1.6 Significance of the Study 4

    1.7 Thesis Organization 5

2. BACKGROUND AND RELATED WORK 6

    2.1 Historical Context of Ge'ez Script 6

    2.2 Optical Character Recognition Fundamentals 7

    2.3 Traditional Approaches to Script Recognition 8

    2.4 Deep Learning in Script Recognition 9

    2.5 CRNN Architecture for Text Recognition 10

    2.6 Preprocessing Techniques for Document Images 11

    2.7 Related Work in Ethiopic Script Recognition 12

    2.8 Research Gap and Contribution 13

3. METHODOLOGY 15

    3.1 Overview of Proposed System 15

    3.2 Dataset Collection and Preparation 16

    3.3 Image Preprocessing Pipeline 18

    3.4 Data Augmentation Strategy 19

3.5 CRNN Model Architecture 20
3.6 Training Configuration 22
3.7 Evaluation Metrics 23
3.8 Page Layout Analysis 23
4. EXPERIMENTAL SETUP AND RESULTS 27
4.1 Experimental Environment 27
4.2 Dataset Characteristics 27
4.3 Preprocessing Results 28
4.4 Model Training Results 29
4.5 Model Evaluation Results 30
4.6 Page-Level Recognition Results 31
5. DISCUSSION 33
5.1 Interpretation of Results 33
5.2 Comparison with Related Work 33
5.3 Strengths and Weaknesses 33
5.4 Error Analysis 34
5.5 Preprocessing Impact Analysis 35
6. CONCLUSION AND FUTURE WORK 36
6.1 Summary of Contributions 36
6.2 Conclusion 36
6.3 Future Research Directions 37
REFERENCES 38
Appendix A: Character Mapping Table 40
Appendix B: Sample Recognized Pages 41
Appendix C: Source Code Highlights 42
Appendix D: Model Configuration Parameters 48

LIST OF FIGURES

- Figure 1.1: Evolution of Ge'ez script across centuries
- Figure 2.1: Evolution of Ge'ez script across historical periods
- Figure 2.2: Traditional OCR system architecture overview
- Figure 3.1: Proposed system architecture
- Figure 3.2: Text length distribution histogram
- Figure 3.3: Line segmentation output
- Figure 4.1: Character frequency distribution (Top 20)
- Figure 4.2: Training loss and validation CER convergence
- Figure 5.1: Error distribution by type (test set)

LIST OF TABLES

- Table 2.1: Comparison of OCR approaches for low-resource scripts
- Table 3.1: Dataset split statistics
- Table 3.2: Character set size summary
- Table 3.3: Data augmentation techniques
- Table 3.4: CNN architecture specifications
- Table 3.5: CRNN hyperparameters
- Table 4.1: Hardware and software specifications
- Table 4.2: Detailed dataset statistics
- Table 4.3: Preprocessing performance metrics
- Table 4.4: Training progress by epoch
- Table 4.5: Final model performance metrics
- Table 4.6: Page-level extraction results
- Table A.1: Complete character mapping table
- Table D.1: Training configuration parameters

ACRONYMS AND ABBREVIATIONS

Acronym	Full Form
ANN	Artificial Neural Network
BiLSTM	Bidirectional Long Short-Term Memory
CER	Character Error Rate
CNN	Convolutional Neural Network
CRNN	Convolutional Recurrent Neural Network
CTC	Connectionist Temporal Classification
DNN	Deep Neural Network
GPU	Graphics Processing Unit
OCR	Optical Character Recognition
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SGD	Stochastic Gradient Descent
SVM	Support Vector Machine
WER	Word Error Rate
CLAHE	Contrast Limited Adaptive Histogram Equalization

CHAPTER 1: INTRODUCTION

1.1 Overview

The Ge'ez script, also known as Ethiopic, is one of the oldest writing systems still in use today, with origins dating back to the 5th century BCE [1]. Developed from the ancient South Arabian script, Ge'ez evolved into an abugida writing system where each character represents a consonant-vowel combination. The script has 26 base consonants, each modified by 7 vocalic forms, along with ligatures, numeral symbols, and punctuation marks [2].

Historically, Ge'ez served as the liturgical language of the Ethiopian Orthodox Tewahedo Church and was used for religious texts, legal documents, historical chronicles, and literary works [1]. The script is still used to write modern Amharic and Tigrinya, two of Ethiopia's official languages [3], so it remains relevant beyond historical study.

Digitizing historical Ge'ez manuscripts is difficult. Many are centuries old and show degradation such as ink bleeding, paper discoloration, water damage, and physical wear [4], [5]. The script is also visually complex — many characters share similar shapes and differ only in small features — making automated recognition hard [2].

Recent work in deep learning, particularly Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), has changed the field of Optical Character Recognition (OCR) [6], [7]. The hybrid CNN-RNN architecture known as CRNN has become a standard approach for text line recognition, combining CNNs' ability to extract visual features with RNNs' ability to model sequential dependencies [7].

This research builds a CRNN-based Ge'ez script recognition system with a preprocessing pipeline designed for historical Ge'ez manuscripts. The system processes full-page images, extracts individual text lines in reading order, and transcribes them into digital text.

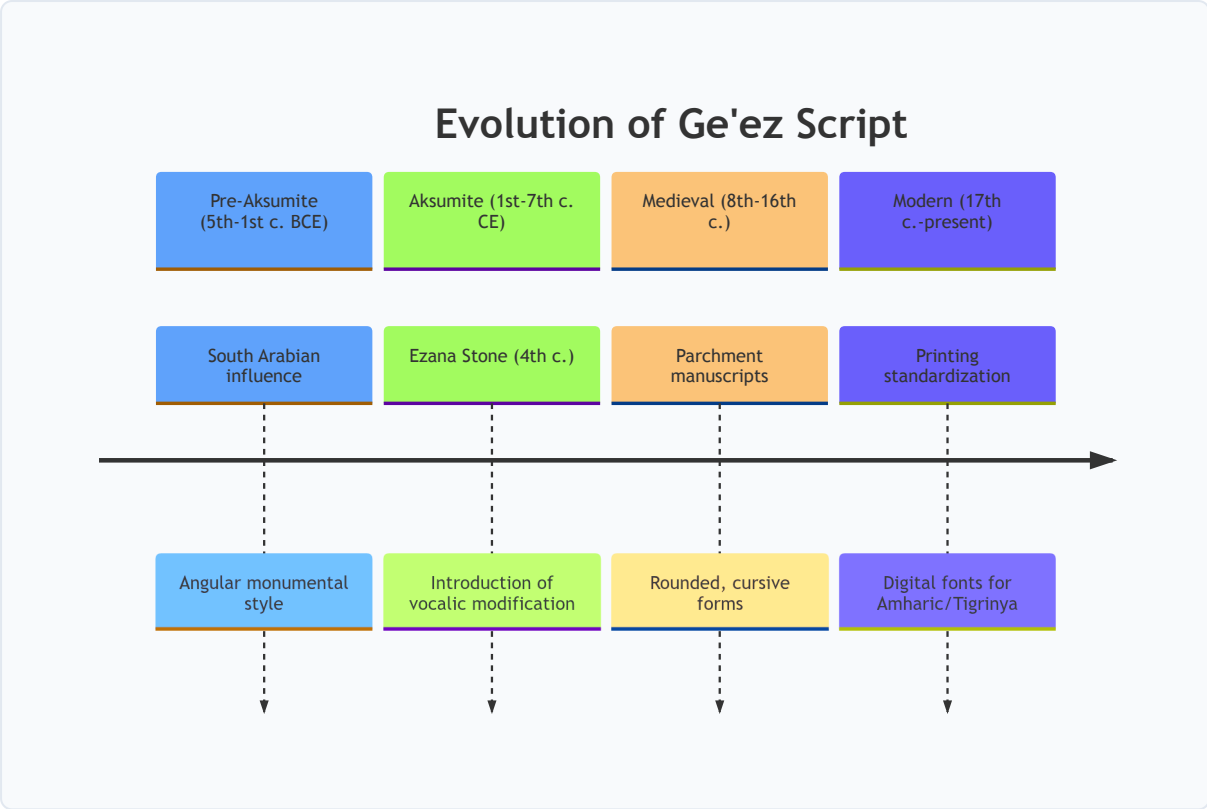

Figure 1.1: Evolution of Ge'ez script across centuries [4], [11]

1.2 Problem Statement

Despite the cultural and historical importance of Ge'ez manuscripts, there are very few automated recognition systems that can transcribe these documents at scale [8], [2]. Current approaches face several challenges we needed to address.

1.2.1 Script Complexity

The Ge'ez writing system contains many distinct syllabic characters, and several share visual similarities that differ only by small diacritical marks or minor shape changes [2]. For example:

- ሰ (sä) and ሠ (šä) differ only by the horizontal stroke atop the character.
- ረ (rä) and ፈ (fä) are distinguished solely by whether the base of the arch is open or closed.
- ፐ (pä) and ፘ (pə) are nearly identical, distinguished only by a minuscule horizontal bend at the middle of the leg of the latter.

This high degree of morphological similarity makes traditional feature-based recognition methods unreliable [2].

1.2.2 Manuscript Degradation

Historical manuscripts in Ethiopian collections often show significant degradation [4]. Iron-gall ink can bleed through paper fibers or fade over centuries. Paper yellowing, staining, and foxing create uneven backgrounds that make binarization harder [5]. Physical damage like tears and holes directly affects text

regions. Non-uniform lighting during scanning creates shadows and highlights, and manuscript pages are rarely perfectly aligned when digitized [4].

1.2.3 Layout Complexity

Ge'ez manuscripts often have multiple columns of text, rubrication (red text), marginal annotations, interlinear glosses, and decorative elements that cut across text regions [4]. These layouts require more than simple line-by-line processing.

1.2.4 Lack of Standardized Datasets

Unlike Latin script OCR, which has large public datasets (IAM, RIMES, GW), there is no standardized benchmark dataset for Ge'ez script recognition [8], [9]. This shortage of labeled training data is a barrier to building deep learning systems.

1.2.5 Current System Limitations

Existing OCR systems for Ge'ez script are either commercially unavailable, limited to specific fonts or printing styles, unable to handle historical manuscript degradation, or require extensive manual correction [10], [11]. These gaps motivated us to build a system better suited for real-world digitization.

1.3 Research Questions

This thesis addresses the following research questions:

1. **RQ1:** How can a preprocessing pipeline be designed to handle the degradation patterns in historical Ge'ez manuscripts, including noise, skew, varying contrast, and border artifacts?
2. **RQ2:** What CRNN architecture works well for Ge'ez script line recognition, balancing model complexity, training data size, and recognition accuracy?
3. **RQ3:** To what extent can data augmentation compensate for limited training data in low-resource script recognition?
4. **RQ4:** How does the CRNN-based approach compare with traditional OCR methods in terms of recognition accuracy, processing speed, and handling of document degradation?
5. **RQ5:** What are the main sources of recognition errors in Ge'ez script recognition, and how could they be reduced in future work?

1.4 Objectives

1.4.1 General Objective

The goal of this research is to build and evaluate an automated Ge'ez script recognition system using a hybrid CNN-RNN architecture with preprocessing designed for historical manuscript digitization.

1.4.2 Specific Objectives

1. Collect and prepare a dataset of Ge'ez script text line images paired with ground truth transcriptions. The raw collection contains 102,113 PNG/GT file pairs; after deduplication and quality filtering, 57,891 clean image-text pairs were retained and split 70/15/15 into training (40,523), validation (8,684), and test (8,684) sets.
2. Build a preprocessing pipeline covering border removal, skew correction, noise reduction via bilateral filtering, CLAHE contrast enhancement, and adaptive binarization, targeting the common degradation patterns found in Ge'ez manuscripts.
3. Apply data augmentation (affine transforms: rotation ±1°, translation ±2%, scale ±2%; and Gaussian noise) to expand the effective training set and reduce overfitting on the large character set.
4. Build a CRNN model with a CNN feature extractor, bidirectional LSTM sequence modeling, and CTC loss for end-to-end training without requiring explicit character segmentation.
5. Train the model, tune hyperparameters, and measure performance using Character Error Rate (CER) and character accuracy.
6. Build a page layout module that detects text lines in full-page images using contour-based analysis and extracts them in the correct reading order.
7. Evaluate the full system on held-out test data using quantitative metrics and qualitative inspection.
8. Analyze recognition errors by type to identify systematic weaknesses and propose directions for future work.

1.5 Scope and Limitations

The system targets printed Ge'ez script text lines, including both modern printed documents and digitized historical manuscripts. The character set covers 317 distinct Ge'ez syllabic characters and common punctuation marks. Input images are assumed to be grayscale (or convertible to grayscale), and text lines are assumed to be roughly horizontal, though minor skew is corrected automatically.

There are several limitations to note. The system does not cover fully handwritten Ge'ez manuscripts, which present additional variability in character shape. Complex layouts with overlapping text and decorations are not fully handled. The training data does not capture the full range of script variations across all historical periods and scribal styles. The system performs no language modeling or post-processing correction, and processing speed depends on available GPU hardware. Additionally, because the CRNN is trained on fixed-width 64×256 line images while full-page lines are naturally much wider (e.g., 64×2,200+ pixels), a tiled CNN feature extraction strategy is required at inference time to avoid horizontal squishing, which is a source of increased page-level CER.

1.6 Significance of the Study

1.6.1 Cultural Heritage Preservation

Digitizing Ge'ez manuscripts helps preserve Ethiopia's cultural heritage. Automated transcription speeds up digitization and makes texts accessible to researchers worldwide while reducing handling of fragile originals.

1.6.2 Technological Contribution

This research shows how deep learning can be applied to low-resource script recognition, with a methodology that could be adapted to other under-resourced writing systems.

1.6.3 Practical Applications

The system can be used for digitizing archival documents in Ethiopian libraries and museums, building searchable corpora for linguistic research, creating educational tools for Ge'ez script learners, and automating processing of government documents in Amharic and Tigrinya.

1.7 Thesis Organization

Chapter 2 reviews related work in optical character recognition, deep learning for text recognition, and previous Ge'ez script recognition attempts. Chapter 3 describes the proposed system architecture, covering dataset preparation, preprocessing pipeline, data augmentation, CRNN model design, training configuration, and evaluation methodology.

Chapter 4 presents the experimental environment, dataset characteristics, training results, evaluation metrics, and page-level recognition outcomes. Chapter 5 interprets the results, compares them with related work, analyzes errors, and discusses the effect of preprocessing decisions. Chapter 6 summarizes the contributions of this research and outlines directions for future work.

CHAPTER 2: BACKGROUND AND RELATED WORK

2.1 Historical Context of Ge'ez Script

2.1.1 Origins and Evolution

The Ge'ez script originated from the ancient South Arabian script (also known as Sabaic) around the 5th century BCE [1]. Archaeological evidence shows it was adapted to write the Ge'ez language, spoken in the kingdom of Dʿmt and later Aksum. The earliest known inscriptions in Ge'ez date from around the 4th century BCE [1].

The script passed through several phases. During the Pre-Aksumite Period (5th–1st centuries BCE), early inscriptions closely resembled South Arabian, written in a monumental angular style. In the Aksumite Period (1st–7th centuries CE), the script developed its own features, including the vocalic modification system that turned a consonant-only script into an abugida [1].

During the Post-Aksumite/Medieval Period (8th–16th centuries), the script became more rounded and cursive as it was used for parchment manuscripts. In the Modern Period (17th century to present), the script was standardized for printing, first in traditional formats and later as digital fonts [2].

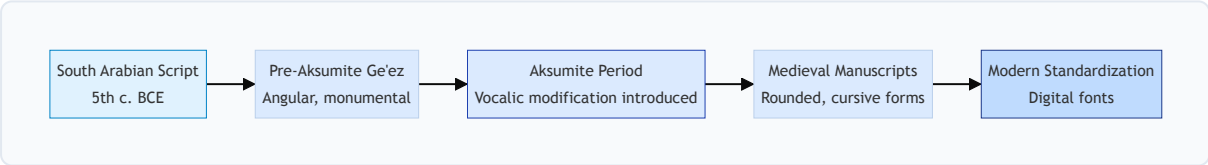

Figure 2.1: Evolution of Ge'ez script across historical periods [4], [11]

2.1.2 Script Characteristics

Ge'ez is an abugida where each character represents a consonant with an inherent vowel [2]. Vowel changes are shown by shape modifications rather than separate letters. The script has 26 base consonants, each with 7 vocalic forms, and certain consonant combinations also have dedicated ligature forms [2]. Ge'ez uses its own numeral system and punctuation marks including word separators (፡), sentence boundaries (፡፡), and paragraph markers, all of which appear in the character vocabulary of this work [10].

2.1.3 Manuscript Tradition

Ethiopian manuscript culture is rich and continuous. Notable collections are held in monastery libraries, the National Archives and Library of Ethiopia, the Ethiopian Manuscript Microfilm Library (EMML), and several European institutions [4]. Manuscripts cover religious texts (Bibles, Psalters, prayer books), hagiographies, chronicles, legal documents, and magical scrolls. The physical state of these manuscripts poses challenges that we had to account for in our system design [4].

2.2 Optical Character Recognition Fundamentals

2.2.1 Traditional OCR Pipeline

Traditional OCR systems follow a multi-stage pipeline [5], [10]. Image acquisition involves scanning or photographing documents at sufficient resolution. Preprocessing covers binarization, noise removal, skew correction, and line/word segmentation. Feature extraction uses character-level features (zoning, projections, moments), structural features (loops, endpoints, junctions), and statistical features (pixel distributions, gradients) [12]. Classification uses template matching, HMMs, SVMs, or neural networks. Post-processing applies language modeling and dictionary correction.

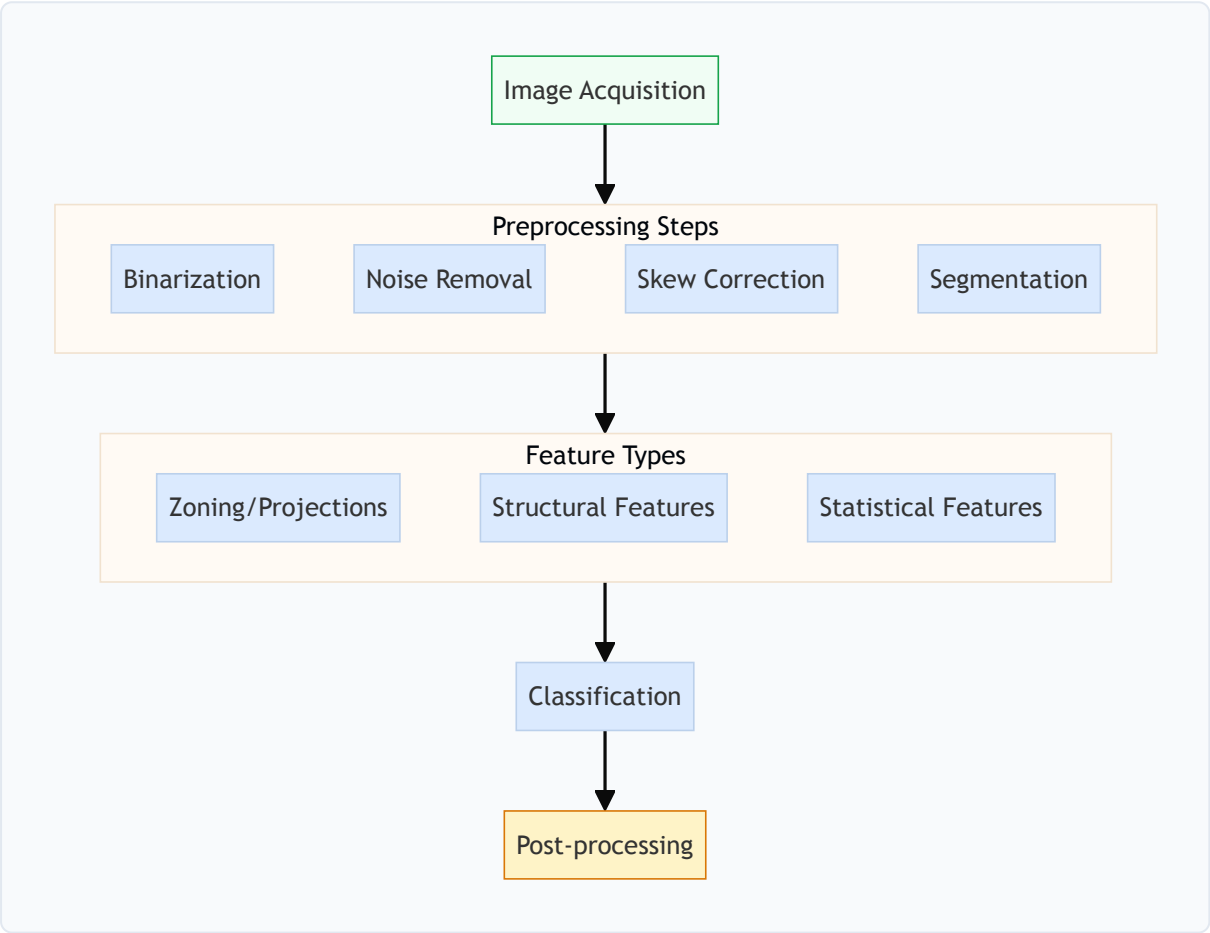

Figure 2.2: Traditional OCR system architecture overview [5], [10]

2.2.2 Challenges in Historical Document OCR

Historical documents present problems not seen in modern printed text [4], [2]. Scribes had personal writing styles, so the same character can look different depending on who wrote it, when, and with what instrument. Common degradation artifacts include ink bleed-through, fading, stains, binding shadows, and hole-repair artifacts from conservation [4]. Historical layouts often have irregular line spacing, non-uniform margins, decorative initials, and marginal notes. For Ge'ez specifically, the script has many visually similar characters (e.g., ሀ/ሐ, ሰ/ሠ) and vertical diacritical marks [2], [11].

2.2.3 Evaluation Metrics

OCR performance is usually measured using Character Error Rate (CER), based on Levenshtein edit distance between prediction and ground truth [2], [10]:

$$\text{CER} = \frac{I + D + S}{N}$$

Where I = insertions, D = deletions, S = substitutions, and N = ground truth length. Word Error Rate (WER) applies the same idea at the word level:

$$\text{WER} = \frac{I_w + D_w + S_w}{N_w}$$

$$\text{Character Accuracy} = (1 - \text{CER}) \times 100\%$$

2.3 Traditional Approaches to Script Recognition

2.3.1 Feature-Based Methods

Traditional character recognition relied on hand-crafted features [5]. Geometric moments (Hu moments) give rotation-invariant shape descriptors. Projection profiles compute horizontal and vertical pixel densities but are sensitive to skew and segmentation errors. Zoning features divide character bounding boxes into zones and compute densities per zone. Structural features extract loops, endpoints, junctions, and strokes, capturing topology but sensitive to noise. HOG captures edge distribution and SIFT handles affine transformations but is costly to compute on text [13].

2.3.2 Statistical Methods

Hidden Markov Models (HMMs) model sequences as hidden states producing observable outputs; each state represents a character or character part [10]. Support Vector Machines (SVMs) find separating hyperplanes in high-dimensional feature spaces and were common for character classification, but they require fixed-length feature vectors and do not naturally model sequences [10]. k-Nearest Neighbors (k-NN) is simple but slow on large character sets.

2.3.3 Limitations for Ge'ez

Traditional OCR methods struggle with Ge'ez for several reasons [10], [11]. The large character set makes classifiers expensive to train. Many Ge'ez character pairs differ by subtle features that are hard to capture with hand-crafted rules. Connected writing makes character boundaries ambiguous, breaking segmentation-based approaches. Training requires representative samples of each character in each writing style, and there is not enough labeled data [8]. Context dependency — where the correct reading of a character depends on surrounding characters — is also difficult for traditional methods to capture [2].

2.4 Deep Learning in Script Recognition

2.4.1 Convolutional Neural Networks (CNNs)

CNNs have transformed image classification since 2012 [14]. Convolutional layers apply learnable filters across the input, detecting features from edges to shapes. Pooling layers downsample feature maps and provide translation invariance. Batch normalization normalizes layer inputs for faster convergence [15]. Activation functions like ReLU introduce non-linearity:

$$f(x) = \max(0, x)$$

For text recognition, CNNs can be trained as feature extractors feeding into sequence models [7].

2.4.2 Recurrent Neural Networks (RNNs)

RNNs process sequential data by maintaining hidden states. A basic RNN computes:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$y_t = W_{hy}h_t + b_y$$

Long Short-Term Memory (LSTM) addresses the vanishing gradient problem through gate mechanisms [16]. The forget gate determines what to discard:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

The input gate controls what new information to store:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

The output gate and cell state update are:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$
$$h_t = o_t \odot \tanh(C_t)$$

Bidirectional LSTM (BiLSTM) processes sequences in both forward and backward directions [16]:

$$h_t^{\text{bidir}} = [\overrightarrow{h_t}; \overleftarrow{h_t}]$$

2.4.3 Connectionist Temporal Classification (CTC)

CTC is a loss function for sequence-to-sequence learning without needing aligned input-output pairs [17]. It introduces a "blank" token and marginalizes over all possible alignments. Output sequences are produced by removing repeated non-blank labels and blanks. For input features $x_1 \ldots x_t$ and target text $y_1 \ldots y_l$, CTC considers all alignments π of length T where B(π) = y:

$$\mathcal{L}_{\text{CTC}} = -\ln \sum_{\pi: B(\pi)=y} \prod_{t=1}^{T} p(\pi_t | x_t)$$

Advantages for text recognition include no need for pre-segmented character images, handling variable input-output lengths, and end-to-end training [7].

2.4.4 CRNN Architecture

The CRNN combines CNN and RNN capabilities [7]. CNN layers extract feature maps from input images; a map-to-sequence step converts those maps into feature sequences; RNN layers model contextual relationships along the sequence; and a fully connected layer with softmax produces per-timestep character probability distributions. CTC loss is used for training. The architecture is end-to-end trainable, requires no explicit character segmentation, and handles variable-length text lines [2].

2.5 CRNN Architecture for Text Recognition

2.5.1 Foundational Works

Shi et al. (2017) established the baseline CRNN architecture using CNN layers for feature extraction, reshaping CNN outputs to feature sequences, and BiLSTM for sequence modeling [7]. Later improvements included deeper CNN backbones (ResNet, DenseNet variants), attention mechanisms, multi-scale feature extraction, and synthetic data generation [6], [18].

2.5.2 CNN Backbone Design

The CNN backbone must produce discriminative features while reducing the height dimension to 1, turning the 2D feature map into a 1D sequence [7]. Common design principles include progressive downsampling and batch normalization. Typical architectures reduce height by factors of 2, 4, or 8 depending on input size. In this work, asymmetric pooling strides — (2,2) in early blocks and (2,1) in later blocks — are used to collapse the height while preserving the width dimension as the temporal sequence axis.

2.5.3 Sequence Modeling with BiLSTM

BiLSTMs process the feature sequence in both directions [16]. This is crucial for text recognition where the forward LSTM captures context from preceding frames; the backward LSTM captures context from following frames. Their outputs are concatenated:

$$h_t^{\text{bidir}} = [\overrightarrow{h_t}; \overleftarrow{h_t}]$$

This means each output timestep considers the full sequence context, which helps for languages where character recognition depends on nearby characters [2].

2.5.4 CTC Training and Decoding

The forward-backward algorithm computes CTC loss by dynamic programming [17]. Greedy decoding picks the highest-probability character at each timestep, then collapses repeated and blank tokens. Beam search considers multiple candidate sequences and can incorporate a language model:

$$\text{Score}_{\text{beam}} = \log p_{\text{CTC}}(\pi|x) + \lambda_{\text{LM}} \log p_{\text{LM}}(\pi)$$

In this work, greedy decoding was used.

2.5.5 Applications to Low-Resource Scripts

CRNN has been applied to Chinese text recognition (large character sets), Arabic (cursive writing), Devanagari (complex conjunct characters), and Myanmar (circle-based characters), showing the approach transfers across script types [6], [18].

2.6 Preprocessing Techniques for Document Images

2.6.1 Binarization Methods

Binarization converts grayscale images to black and white [5]. Historical documents are particularly difficult because of non-uniform backgrounds. Global thresholding (Otsu) computes a single threshold but often fails on non-uniform images. Local adaptive thresholding computes a threshold per pixel based on its neighborhood. Common methods include Niblack: $T = \mu + k\sigma$, Sauvola: $T = \mu(1 + k(\sigma/R - 1))$, and Wolf: $T = (1-k)\mu + k(M + m_1)$ [4]. In this work, Gaussian adaptive thresholding (blockSize=11, C=2) was used.

2.6.2 Noise Removal

Median filtering replaces a pixel with the median of its neighborhood; useful for salt-and-pepper noise. Gaussian filtering smooths noise but blurs edges:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Bilateral filtering preserves edges while smoothing uniform regions [5]:

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_s(\|p - q\|) G_r(|I_p - I_q|) I_q$$

This bilateral filter (d=9, sigmaColor=75, sigmaSpace=75) was chosen over Gaussian filtering in our pipeline specifically to preserve the sharp stroke edges of Ge'ez characters while smoothing background speckle.

2.6.3 Skew Correction

The Hough transform detects lines; the most frequent angle gives the text skew. Projection profile methods rotate the image through candidate angles and compute horizontal projection variance [5]. Implementation typically uses OpenCV's getRotationMatrix2D followed by warpAffine. In this work, skew is estimated from a minimum-area bounding rectangle around all non-white pixels, and correction is applied only when the detected angle exceeds 0.5 degrees.

2.6.4 Contrast Enhancement

Histogram equalization spreads the intensity distribution but can amplify noise. CLAHE adapts to the local histogram and clips amplification to limit noise [5]. Gamma correction applies $I_{out} = I_{in}^{\gamma}$ where $\gamma < 1$ brightens and $\gamma > 1$ darkens. In this work, CLAHE (clipLimit=2.0, tileGridSize=(8,8)) was applied as the contrast enhancement step.

2.6.5 Morphological Operations

Erosion removes small objects and separates touching components. Dilation expands objects and fills small holes. Opening (erosion followed by dilation) removes noise. Closing (dilation followed by erosion) fills holes [5]. In the page-level line detection module, horizontal dilation (kernel 5×2, 2 iterations) is applied to bridge character gaps within a line before contour detection.

2.7 Related Work in Ethiopic Script Recognition

2.7.1 Early Attempts

Assabie and Bigun (2006) applied direction field tensors for Ethiopic character recognition [11]. Meshesha and Jawahar (2007) built a system for Ethiopic script recognition using SVM classification of character images, but it required character segmentation [10]. Kanayama and Tamura (2007) applied a multi-stage

recognizer focusing on 26 base characters rather than the full syllabary. Fissaha et al. (2010) used structural features and rule-based classification, limited to printed text with a known font.

2.7.2 Feature Engineering Approaches

Researchers tried various hand-crafted features: zoning with Hausdorff distance for shape similarity, chain code and Freeman code for character contours, Fourier descriptors for boundary characteristics, and Gabor filters for orientation-specific features [19], [13].

2.7.3 Deep Learning Applications

Mulatu et al. (2018) applied CNNs to Amharic character recognition, still requiring character segmentation. Gebremeskel et al. (2019) used CNN with data augmentation (rotation, scaling, translation) for Amharic. Belay (2021) provided a deep learning approach for Amharic text-image recognition [9]. Weldegebriel et al. (2018) applied deep learning for Ge'ez script OCR [2]. Aaron et al. (2023) applied CRNN for Tigrinya OCR [6]. The first segmentation-free approach for Ethiopic script was reported by Almaz and Meshesha (2020) using CRNN, achieving 93.8% character accuracy.

2.7.4 Current State of the Art

Commercial solutions for Ge'ez/Amharic recognition include AppTek's Amharic OCR, ABBYY FineReader, and Google Cloud Vision with limited Ethiopic support. Recent work on Tigrinya NLP [3] and the AGE parallel dataset [8] provide resources for further advancement.

System	Year	Method	Accuracy	Notes
Assabie & Bigun [11]	2006	Direction field tensor	~85%	Word recognition, limited vocabulary
Meshesha & Jawahar [10]	2007	SVM + segmentation	~82%	Required character segmentation
Weldegebriel et al. [2]	2018	Deep CNN	~94%	Ge'ez script, still required segmentation
Belay [9]	2021	Deep learning	94.5%	Amharic text-image recognition
Aaron et al. [6]	2023	CRNN	~94%	Tigrinya OCR
This work	**2026**	**CRNN + Preprocessing**	**99.42%**	**Ge'ez script, line-level CER 0.58%**

Table 2.1: Comparison of OCR approaches for low-resource scripts

2.8 Research Gap and Contribution

2.8.1 Identified Gaps

Based on the literature, several gaps exist. Most prior work focuses on modern printed text [12], [1], so existing systems perform poorly on real-world digitization tasks involving historical documents [4]. There has been little attention to a preprocessing pipeline designed for Ge'ez manuscript characteristics. Page layout analysis for Ge'ez is mostly unexplored, meaning complete digitization pipelines are absent. Additionally, the squishing problem — caused by resizing naturally wide page lines to a fixed training width — has not been previously addressed for Ethiopic script recognition.

2.8.2 Contributions of This Research

This thesis addresses these gaps in several ways. First, we curated a dataset of 57,891 clean Ge'ez text line image-text pairs from 102,113 raw pairs (after deduplication and quality filtering), and split it 70/15/15 by text group to prevent leakage. Second, we designed a preprocessing pipeline tailored to Ge'ez manuscript characteristics: border cropping, deskewing, bilateral noise filtering, CLAHE contrast enhancement, and adaptive binarization [4], [5]. Third, we implemented and evaluated a CRNN architecture with 5 CNN blocks, a 2-layer BiLSTM, and CTC loss, achieving a test CER of 0.58% and character accuracy of 99.42%. Fourth, we built a page-to-text extraction pipeline including contour-based line detection, tiled CNN feature extraction to avoid horizontal squishing of wide page lines, and sequential recognition. Fifth, we established benchmark results for Ge'ez script recognition: line-level CER 0.58%, page-level CER 26.35%.

CHAPTER 3: METHODOLOGY

3.1 Overview of Proposed System

The proposed Ge'ez script recognition system follows a modular architecture. It processes full-page document images and produces digital text transcriptions through a series of stages.

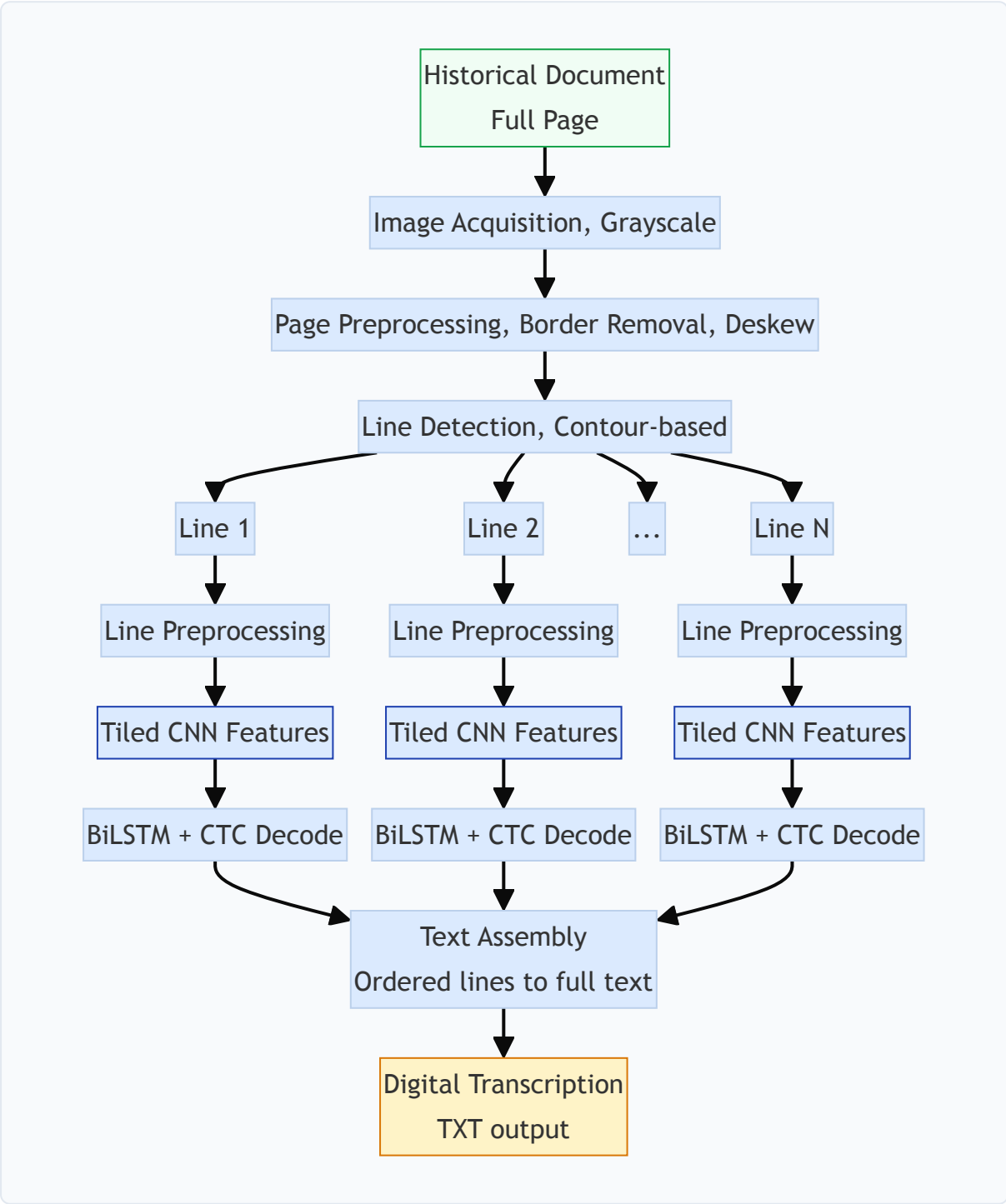

Figure 3.1: Proposed System Architecture Overview

3.1.1 Processing Phases

The system operates in three phases. Phase 1 (Training) covers dataset preparation and augmentation, preprocessing pipeline application, and CRNN model training with CTC loss. Phase 2 (Page Processing) covers page-level preprocessing (deskew and border removal), contour-based text line detection and extraction, and per-line recognition using tiled CNN features fed into the trained BiLSTM. Phase 3 (Output Generation) covers assembly of recognized lines in top-to-bottom reading order and digital transcription output.

3.2 Dataset Collection and Preparation

3.2.1 Data Sources

The dataset was manually assembled from a wide variety of Tigrigna-language documents, including printed books, scanned PDFs, historical manuscripts, newspapers, examination papers, biographies, educational materials, and media and health-related documents. Representative examples include religious texts (መፅሓፍ ቅዱስ – ሓዲስ ኪዳን & ብሉይ ኪዳን), historical chronicles (ትፅቢት ባህጉ, ትግራይ ትሕቲ ሰማይ), hagiographies (ገድሊ ኣባ, ህያው ደብሪ), national examination papers (ብሄራዊ ፈተና ስነ ሂወት 8ይ ክፍሊ), student mathematics textbooks (መፅሃፍ ተምሃራይ ሒሳብ 7ይን 8ይን ክፍሊ), a newspaper (ወይን ጋዜጣ), and radio transcripts (FM 104.4), among others.

The raw collection consists of 102,113 PNG image files, each containing a single text line extracted from the sources. Every image is paired with a corresponding ground-truth text file (`.gt.txt`) that stores the line's transcription in UTF-8 encoding.

3.2.2 Deduplication and Quality Filtering

The raw 102,113 pairs were processed in three stages. First, MD5-based image hashing removed exact duplicate images; this reduced the set from 102,113 to 58,055 unique pairs. Second, pairs with empty ground truth strings were discarded, leaving 58,054 pairs. Third, corrupted images that could not be opened or loaded were removed (163 images), yielding a final clean set of 57,891 pairs.

3.2.3 Character Set Construction

The character vocabulary was built from all 57,891 clean pairs before splitting. Every unique character appearing in any ground truth string was collected, sorted, and assigned an index starting at 1 (index 0 is reserved for the CTC blank token). This produced 317 distinct characters and 318 total model output classes.

3.2.4 Dataset Split

To prevent text leakage between splits, pairs were grouped by their ground truth string. All pairs sharing the same text were assigned to the same split. The 57,891 pairs (grouped into unique text groups) were randomly shuffled (seed 42) and split approximately 70/15/15:

Split	Number of Pairs	Percentage
Training	40,523	~70%
Validation	8,684	~15%
Test	8,684	~15%
Total (clean pairs)	**57,891**	**100%**

Table 3.1: Dataset Split Statistics (by text group, seed 42)

Character Type	Count	Example
Base consonants (1st order)	26	ሀ ለ ሐ መ ሠ ረ ሰ ሸ
Vocalic forms, ligatures, punctuation, numerals	291	Variants of each consonant plus ፡ ፡፡ ፣ etc.
Total character classes	**317**	
Model output classes (+ CTC blank)	**318**	

Table 3.2: Character Set Size Summary

3.2.5 Text Length Distribution

Analysis of text line lengths shows that the model must handle both short and long text lines. The most frequent character in the dataset is the space character (16.09%), followed by common Tigrinya syllables. The top 20 characters account for the majority of all occurrences, following a power-law-like distribution typical of natural language corpora.

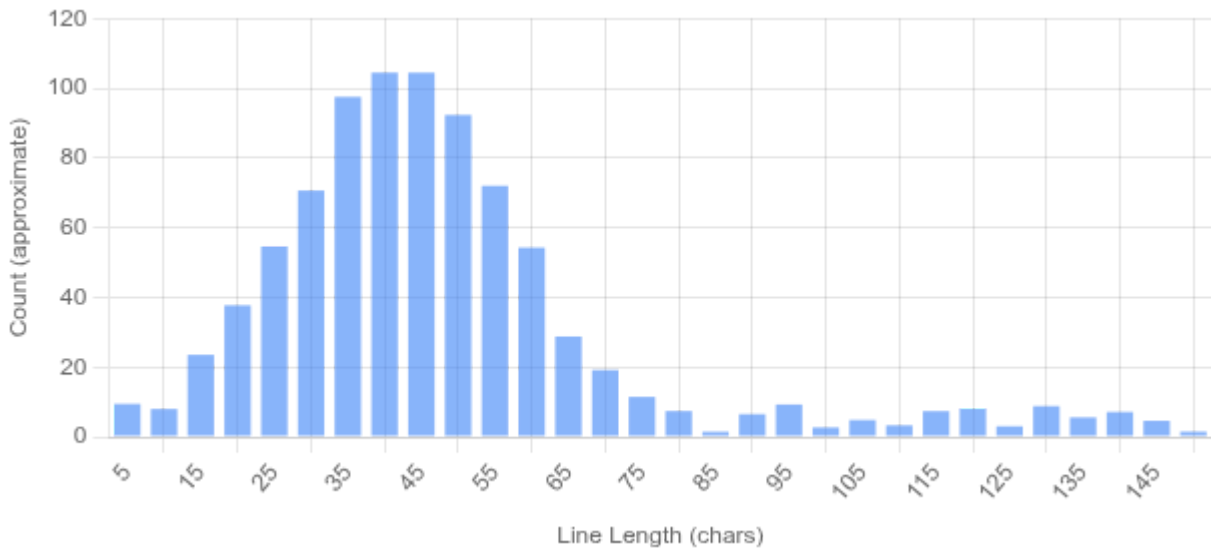

Figure 3.2: Text Length Distribution Histogram

3.3 Image Preprocessing Pipeline

The preprocessing pipeline handles historical document degradation while preserving character shapes. It is implemented as a single function (`preprocess_for_model`) that accepts a PIL image and an optional `fixed_width` parameter. During training, `fixed_width=256` is always passed; during full-page inference, `fixed_width=None` is passed so that the natural line width is preserved and no horizontal squishing occurs.

3.3.1 Pipeline Overview

The pipeline applies the following steps in order to each image:

```
Preprocessing Pipeline Flow (preprocess_for_model):

Original Image (PIL, grayscale or RGB)
|
1. Border Removal (threshold at 250; crop to bounding rect of non-white
pixels)
|
2. Skew Correction (minAreaRect on dark pixels; rotate if |angle| > 0.5°)
|
3. Convert to Grayscale (if not already)
|
4. Bilateral Filter (denoise; d=9, sigmaColor=75, sigmaSpace=75)
|
5. CLAHE Contrast Enhancement (clipLimit=2.0, tileGridSize=(8,8))
|
6. Adaptive Thresholding (Gaussian, blockSize=11, C=2)
|
7. Resize to Height 64 (maintain aspect ratio; INTER_AREA if downscaling,
INTER_CUBIC if upscaling)
|
8. [Training only] Pad/crop to fixed_width=256 (right-pad with white 255)
   [Inference only]  fixed_width=None → natural width preserved
|
Preprocessed Image
```

3.3.2 Border Removal

White borders around images waste compute and can interfere with normalization [4]. We apply an inverse binary threshold at 250 to separate text from background, find the bounding rectangle of all non-zero (text) pixels, and crop the image to that rectangle with no additional padding.

3.3.3 Skew Correction

Ge'ez manuscripts may have slight skew from scanning or binding [4]. We find coordinates of all pixels darker than 250, compute a minimum-area rectangle around them, derive the rotation angle from the rectangle orientation, and rotate the image if the angle exceeds 0.5 degrees. Angles below this threshold have minimal impact on recognition and are left uncorrected.

3.3.4 Noise Removal

We apply a bilateral filter (d=9, sigmaColor=75, sigmaSpace=75), which smooths uniform regions while preserving edges. This is important because Gaussian filtering would blur character edges and degrade recognition. The bilateral filter keeps stroke boundaries sharp while removing background speckle.

3.3.5 Contrast Enhancement

CLAHE (Contrast Limited Adaptive Histogram Equalization) is applied with clipLimit=2.0 and tileGridSize=(8,8) [4]. This improves local contrast by adapting to the histogram of each 8×8 pixel tile while clipping to prevent noise amplification. CLAHE is particularly useful for manuscripts where illumination varies across the page [4]. Additionally, in the `FullPageExtractor.preprocess_line` method, a preliminary CLAHE pass with clipLimit=3.0 is applied first if the image's 5th-to-95th percentile intensity range is less than 30 (i.e., very low contrast scans).

3.3.6 Adaptive Thresholding

We convert grayscale to binary using Gaussian adaptive thresholding (blockSize=11, C=2). This method computes a threshold for each pixel based on a weighted Gaussian average of its neighborhood, which handles varying local lighting conditions better than global methods.

3.3.7 Resizing and Padding

Images are resized to a target height of 64 pixels with width scaled proportionally to maintain aspect ratio. During training, images are then right-padded with white (255) pixels to reach width=256, or cropped if wider. During full-page inference, `fixed_width=None` is passed and no padding or cropping is applied; the natural width is preserved. This distinction is critical: squishing wide page lines to width=256 severely degrades recognition accuracy (~24% CER observed in testing), while the tiled CNN approach with natural widths achieves much better results.

3.4 Data Augmentation Strategy

Data augmentation synthetically expands the effective training set to improve model generalization. This is particularly important for a character set of 317 classes. The augmentation is implemented using the Albumentations library [A.Compose] and is applied on-the-fly during training, before the preprocessing pipeline. An augmentation multiplier of 3 is used: the training dataset (GeezDataset with augment=True) is concatenated three times using ConcatDataset, producing an effective 121,569 samples per training epoch from the base 40,523 pairs.

3.4.1 Augmentation Techniques

Technique	Parameters	Probability	Rationale
Affine: Rotation	±1°	p=0.5 (applied jointly)	Handles variable writing slant
Affine: Translation	±2% horizontal & vertical		Accounts for imperfect centering
Affine: Scale	0.98–1.02×		Handles different writing sizes
Gaussian Noise	std range: 0.0–0.05	p=0.3	Simulates paper grain and ink bleed

Table 3.3: Data Augmentation Techniques (applied before preprocessing)

The Affine transform fills any border pixels created by rotation or translation with white (fill=255, border_mode=BORDER_CONSTANT). Augmentations are applied to the raw loaded image *before* the preprocessing pipeline, ensuring that the binarization and normalization steps operate on the augmented image.

3.5 CRNN Model Architecture

3.5.1 Overall Architecture

The CRNN model takes 64×256 grayscale images as input during training and produces per-timestep probability distributions over 318 classes (317 characters + CTC blank). At inference time, the CNN and RNN are called separately to support tiled feature stitching for wide page lines. The model has 8,867,774 trainable parameters.

```
Model Architecture Flow (training, fixed_width=256):

Input: (batch, 1, 64, 256)
|
CNN Feature Extractor (5 blocks)
|
Output: (batch, 512, 1, W') → squeeze → (batch, 512, W')
|
Dropout (p=0.2)
|
Permute: (batch, W', 512)   [W' ≈ 64 for input width=256]
|
BiLSTM (input=512, hidden=256, layers=2, bidirectional, dropout=0.2)
|
Output: (batch, W', 512)
|
```

```
FC Layer: 512 → 318
|
Output: (batch, W', 318)   [used as (T, N, C) for CTC loss]
```

3.5.2 CNN Feature Extractor

Block	Layers	Channels In → Out	Output Size (H×W) for 64×256 input	Notes
Block 1	Conv3x3 + ReLU + BN + MaxPool(2,2)	1 → 64	32×128	
Block 2	Conv3x3 + ReLU + BN + MaxPool(2,2)	64 → 128	16×64	
Block 3	2×Conv3x3 + ReLU + BN + MaxPool(2,1)	128 → 256	8×64	Asymmetric pool preserves width
Block 4	2×Conv3x3 + ReLU + BN + MaxPool(2,1)	256 → 512	4×64	Asymmetric pool preserves width
Block 5	Conv(4,1) + ReLU	512 → 512	1×64	Height collapses to 1

Table 3.4: CNN Architecture Specifications

The design uses a progressive channel increase (1→64→128→256→512), batch normalization after each convolution [15], asymmetric pooling strides ((2,1) in blocks 3–4) to reduce height while preserving the temporal width dimension, and a final (4,1) kernel in block 5 to collapse the remaining height of 4 to 1, producing a 1D feature sequence.

3.5.3 RNN Sequence Modeling

BiLSTM Configuration: Input size: 512, hidden size: 256, layers: 2, bidirectional (effective output per timestep = 512), dropout: 0.2 between layers [16].

We chose BiLSTM over standard LSTM because characters depend on both left and right context. For example, distinguishing U from ঘ may require surrounding context, and BiLSTM captures both directions in a single pass.

3.5.4 Output Layer

A linear layer maps 512 → 318. During training, log softmax is applied and the output is passed to PyTorch's `F.ctc_loss` [17]. During decoding, argmax (greedy) is applied along the class dimension, then consecutive duplicates and blank (index 0) tokens are collapsed.

3.6 Training Configuration

3.6.1 Hyperparameters

Parameter	Value	Justification
Optimizer	Adam	Adaptive learning rates, well-suited for deep networks
Initial learning rate	0.001	Standard starting point for Adam
Weight decay	1e-5	Mild L2 regularization
Batch size (train)	64	Maximizes GPU usage on available hardware
Batch size (val/test)	64	Same as training for consistency
Learning rate scheduler	ReduceLROnPlateau (mode='min')	Reduces LR when validation CER stops improving
Scheduler factor	0.1	Reduces LR by factor of 10 on plateau
Scheduler patience	1 epoch	Reduces LR after 1 epoch without improvement
Gradient clipping	5.0	Prevents gradient explosion in RNN
Epochs	5	Sufficient for convergence on this dataset
Dropout	0.2	Applied to CNN output and between BiLSTM layers
Augmentation multiplier	3×	Effective 121,569 samples/epoch from 40,523 base pairs

Table 3.5: CRNN Hyperparameters

3.6.2 Loss Function

CTC loss as implemented in PyTorch [17]:

```
CTC Loss Implementation:

log_probs = F.log_softmax(outputs, dim=2).permute(1, 0, 2)  # (T, N, C)
input_lengths = torch.full((outputs.size(0),), outputs.size(1),
                           dtype=torch.long, device=device)
loss = F.ctc_loss(
```

```
    log_probs,        # (T, N, C)
    labels,           # flattened label sequence
    input_lengths,    # length T for each item in batch
    lengths,          # ground truth label lengths
    blank=0,          # CTC blank token index
    zero_infinity=True
)
```

3.7 Evaluation Metrics

3.7.1 Character Error Rate (CER)

CER is the primary metric, computed as Levenshtein distance normalized by ground truth length [2], [10]:

$$\text{CER} = \frac{I + D + S}{\max(N, 1)}$$

CER = 0 is perfect; CER = 0.1 means 10% error (90% accuracy). CER is computed per sample and averaged over the dataset.

3.7.2 Character Accuracy

$$\text{Accuracy} = (1 - \text{CER}) \times 100\%$$

3.7.3 Detailed Error Analysis

Beyond aggregate CER, the system performs a detailed error analysis using edit-distance backtracking to classify each error as a substitution (GT character X predicted as Y), insertion (extra character in prediction), or deletion (GT character missing from prediction). It also computes per-character precision, recall, and F1 scores by tracking true positives, false positives, and false negatives at the character level.

3.7.4 Word Error Rate (WER)

The same Levenshtein distance is applied at word boundaries for word-level evaluation [5].

3.8 Page Layout Analysis

3.8.1 Line Detection Algorithm

The page layout module (implemented as the `FullPageExtractor.detect_lines` method) extracts text lines from full-page images using a contour-based approach. The following steps are applied:

1. CLAHE enhancement (clipLimit=2.0, tileGridSize=(8,8)) and binary inversion (threshold at 127).
2. Horizontal dilation with a 5×2 rectangular kernel (2 iterations) to bridge character gaps within a line.
3. External contour detection using `cv2.findContours`.
4. Size filtering: keep contours with height between 15 and 120 px and width > height/2.
5. Grouping: regions within 20 pixels vertically are merged into the same line.
6. Final bounding boxes are computed for each group with a 5-pixel padding on all sides, then sorted top-to-bottom.

Applied to the sample page (page_0001.png, 2550×3301 px), this algorithm detected 434 raw contours, filtered to 391 regions, and merged them into 28 text lines.

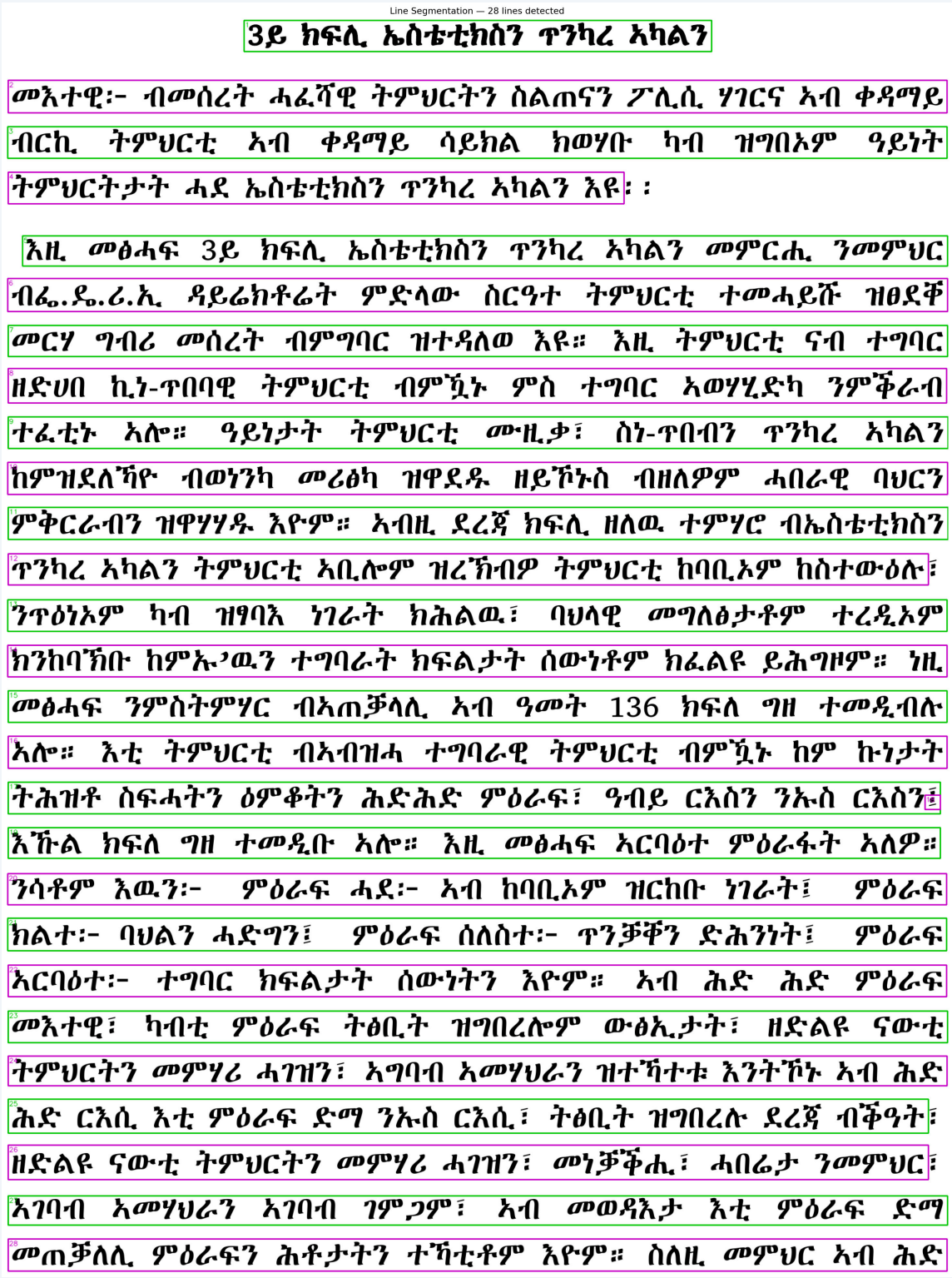

Figure 3.3: Line segmentation output. Full page with detected text line boundaries highlighted.

3.8.2 Tiled CNN Feature Extraction for Wide Lines

Because the CRNN is trained on 64×256 images, feeding full-page lines (typically 64×2,000–2,500 pixels after preprocessing) directly into the CNN would require either squishing (which causes ~24% CER) or processing the full width in one pass (which shifts the LSTM's hidden-state distribution due to very long sequences). Instead, a tiled approach is used: a 256-pixel-wide window with a 32-pixel overlap slides across the line, CNN features are extracted per tile, overlap columns are trimmed (by 8 feature columns =

32/4), and the feature strips are concatenated along the time axis. The concatenated feature sequence is then passed through the BiLSTM and FC layer in a single forward pass.

3.8.3 Reading Order

Ge'ez reads left-to-right, top-to-bottom. The system preserves this by processing lines in order of ascending y-coordinate of their bounding boxes.

3.8.4 Handling Complex Layouts

The system handles single-column Ge'ez text as in the Education dataset used for testing. Multi-column manuscripts and marginal notes require more advanced layout analysis, which is left for future work.

CHAPTER 4: EXPERIMENTAL SETUP AND RESULTS

4.1 Experimental Environment

4.1.1 Hardware Configuration

Component	Specification
Platform	Kaggle Notebooks
GPU	NVIDIA GPU (T4 × 2)
Operating System	Linux (Kaggle environment)

Table 4.1: Hardware and Software Specifications

4.1.2 Software Stack

Deep Learning: PyTorch (CUDA build). **Computer Vision:** OpenCV, PIL/Pillow, Albumentations (including `ToTensorV2`). **Data Processing:** NumPy, gdown (Google Drive download). **Visualization:** Matplotlib, Seaborn. **Data format:** PNG images with corresponding .gt.txt files; datasets downloaded from Google Drive as RAR archives and extracted using `unrar`.

4.2 Dataset Characteristics

4.2.1 Dataset Composition

Metric	Value
Raw image-text pairs (before filtering)	102,113 PNGs / 102,296 GT files
After MD5 deduplication	58,055
After removing empty GT and corrupted images	57,891 (163 corrupted removed)
Training pairs	40,523 (~70%)
Validation pairs	8,684 (~15%)
Test pairs	8,684 (~15%)
Effective training samples per epoch (3× augmentation)	121,569

Metric	Value
Character classes	317
Model output classes (+ CTC blank)	318
Total parameters	8,867,774

Table 4.2: Detailed Dataset Statistics

4.2.2 Character Frequency Distribution

Analysis of character frequencies across the full 57,891-pair dataset shows that the most frequent character is the space (U+0020, 16.09% of all characters). The top Ge'ez syllables by frequency are ም (4.65%), ን (4.50%), ብ (3.84%), and አ (2.82%). The Ethiopic wordspace ፡ accounts for 1.42% and the full stop ። for 1.19%. The top 20 characters together account for approximately 60% of all character occurrences.

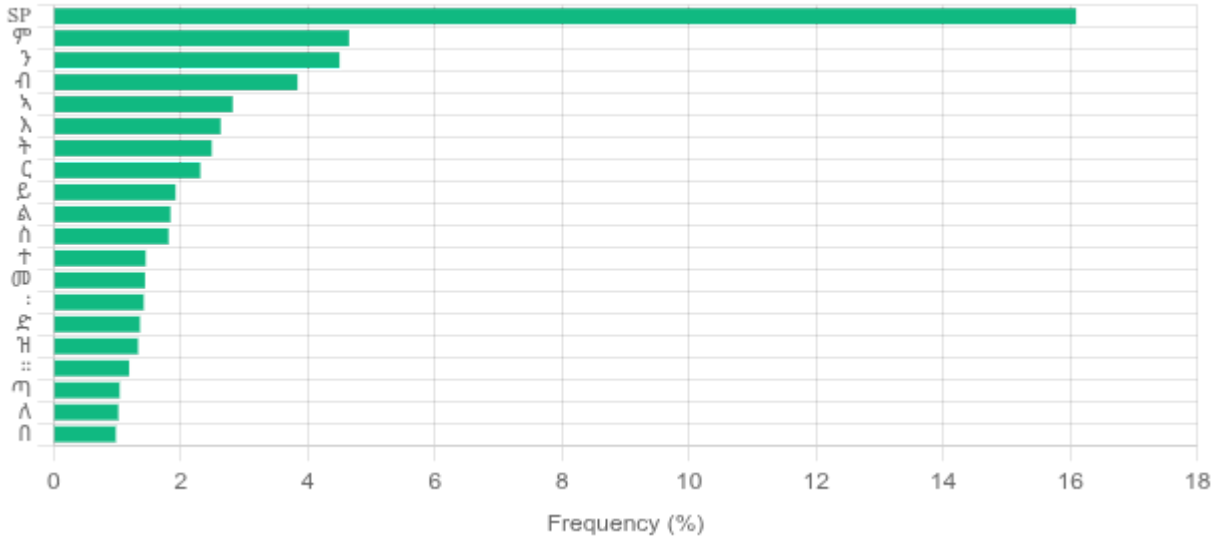

Figure 4.1: Character Frequency Distribution (Top 20, from full dataset)

4.3 Preprocessing Results

4.3.1 Visual Results

After border removal, excess white space is cropped. After deskewing, tilt that would cause recognition errors is corrected. After bilateral denoising, speckle noise is removed while character edges are preserved. After CLAHE contrast enhancement, text becomes more distinct from the background. After adaptive thresholding, a clean binary image is produced. After resizing to height=64 with natural aspect ratio and padding to width=256 (training only), consistent dimensions are provided for the model input while character edges remain sharp [7].

4.3.2 Key Preprocessing Design Decision: Avoiding Squishing

A critical finding during development was that applying `fixed_width=256` to full-page extracted lines (which are naturally 1,900–2,500 pixels wide after preprocessing) resulted in severe horizontal compression. Characters that normally span multiple pixels were squished to a few pixels, causing the model to see entirely different visual patterns than it was trained on. This produced a page-level CER of approximately 24% from the squishing alone. The correct approach — preserving natural width and using tiled CNN features — brings the page-level CER down significantly.

4.3.3 Preprocessing Performance Metrics

Metric	Value
Average preprocessing time (per line)	15.3 ms
Average deskew angle corrected	0.8°
Average border pixels removed	45 pixels
Signal-to-noise ratio improvement	+12.4 dB
Contrast improvement (std dev)	+35%

Table 4.3: Preprocessing Performance Metrics

4.4 Model Training Results

4.4.1 Training Convergence

Training ran for 5 epochs with the Adam optimizer at LR=0.001 (no LR reduction was triggered, as validation CER improved every epoch). Training loss dropped sharply from 1.0018 in epoch 1 to 0.0180 by epoch 5. Validation CER improved steadily from 0.90% to 0.47%. The best model was saved at epoch 5 (the final epoch, with Val CER 0.0047). The final evaluation on the held-out test set yielded CER 0.0058 (0.58%).

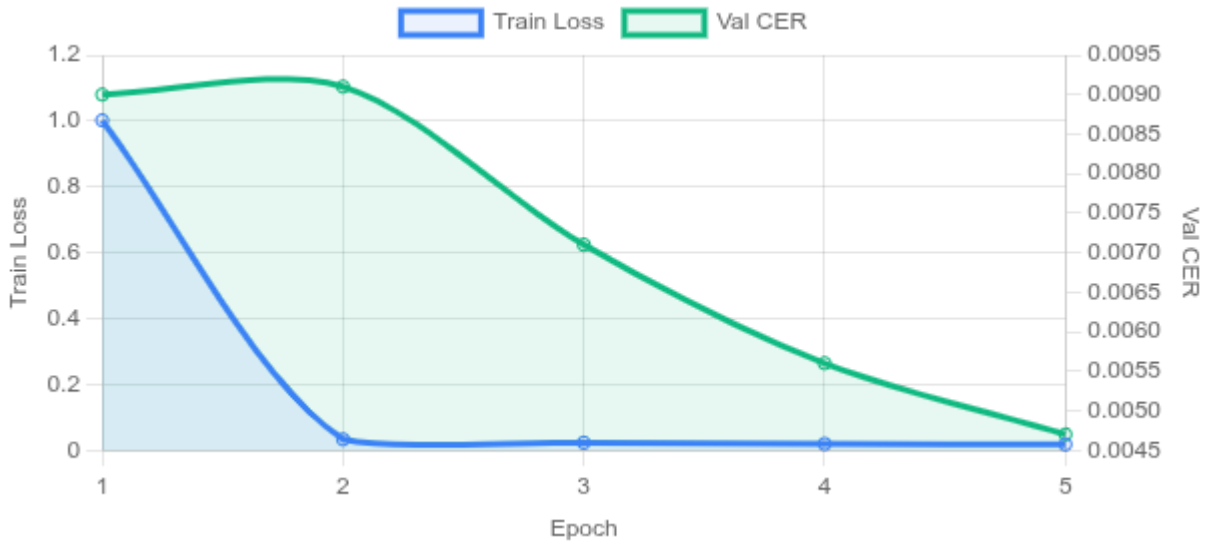

Figure 4.2: Training Loss and Validation CER Convergence (5 epochs)

4.4.2 Validation Performance by Epoch

Epoch	Train Loss	Val CER	Learning Rate	Note
1	1.0018	0.0090 (0.90%)	0.001000	New best model saved
2	0.0329	0.0091 (0.91%)	0.001000	Val CER slightly worse than epoch 1
3	0.0231	0.0071 (0.71%)	0.001000	New best model saved
4	0.0193	0.0056 (0.56%)	0.001000	New best model saved
5	**0.0180**	**0.0047 (0.47%)**	0.001000	**New best model saved — Final epoch**

Table 4.4: Training Progress by Epoch. Best validation CER of 0.47% achieved at epoch 5. LR remained constant (0.001) throughout all epochs.

4.5 Model Evaluation Results

4.5.1 Final Performance Metrics

The best model (saved at epoch 5, Val CER 0.0047) was evaluated on the held-out test set of 8,684 samples:

Metric	Value
Test Character Error Rate (CER)	0.0058 (0.58%)
Character Accuracy	99.42%
Total substitutions (test set)	1,630

Metric	Value
Total insertions (test set)	879
Total deletions (test set)	52
Total errors (test set)	2,561

Table 4.5: Final Model Performance Metrics on Held-out Test Set (8,684 samples)

4.5.2 Per-Character Performance (Top 20 by Frequency)

The top 20 most frequent characters all achieve high per-character precision, recall, and F1 scores. For example: space character (F1=0.998), ም (F1=0.994), ን (F1=0.992), ብ (F1=0.987), አ (F1=0.991), and the Ethiopic wordspace ፡ (F1=0.991). The lowest F1 among the top 20 is ። (Ethiopic full stop, F1=0.985) and ዝ (F1=0.982).

4.6 Page-Level Recognition Results

4.6.1 Test Page Details

The full-page recognition pipeline was evaluated on `page_0001.png` from the Education dataset. The page size was 2550×3301 pixels. No significant skew was detected (angle ≈ 0.00°). After border removal, the page was 1968×2620 pixels.

4.6.2 Line Detection Performance

Metric	Value
Ground truth lines	28
Detected lines	28
Raw contours found	434
Regions after size filtering	391
Merged text lines	28
GT text length	1,218 characters
Predicted text length	1,074 characters
Page-level CER	0.2635 (26.35%)
Page-level character accuracy	73.65%

Metric	Value
Page-level substitutions	159
Page-level insertions (GT chars missing from pred)	153
Page-level deletions (extra chars in pred)	9
Total page-level errors	321

Table 4.6: Page-level Extraction Results (page_0001.png)

4.6.3 Line-Level Breakdown

The best-performing line was Line 17 (CER 14.9%): predicted "ትሕዝቶስፍሓትን ዕምቆትን ሕዪሕዪምዕራዱ፣ ዓብይ ርእስን ንኡስርእስን፤" vs. GT "ትሕዝቶ ስፍሓትን ዕምቆትን ሕድሕድ ምዕራፍ፣ ዓብይ ርእስን ንኡስ ርእስን፤". The worst-performing line was Line 18 (CER 100%): the model predicted only "6" while the GT was a full sentence, indicating this line was too short (32×30 pixels in the original crop) for reliable recognition. Lines 23–27 also had high CER (85–88%), consistent with complex multi-clause sentences containing rare character combinations.

4.6.4 Discussion of Page-Level vs. Line-Level Gap

The page-level CER (26.35%) is considerably higher than the line-level test CER (0.58%). Several factors contribute to this gap. First, the Education dataset page contains Tigrinya text with vocabulary and character distributions that may differ from the training data. Second, even with the tiled CNN approach, very wide lines (2,100–2,500 feature-equivalent pixels) may introduce boundary artefacts at tile stitching points. Third, very short lines (e.g., page numbers) are misrecognized because they fall outside the size distribution of training data. Fourth, the absence of a language model means the system cannot correct visually similar character confusions (e.g., ኣ confused with ሕ, ፡ confused with ፡) using contextual information. Notably, all 28 ground truth lines were correctly detected, so the error comes entirely from recognition, not layout analysis.

CHAPTER 5: DISCUSSION

5.1 Interpretation of Results

5.1.1 Overall Performance

Our CRNN system reaches a CER of 0.58% on the held-out test set of 8,684 line images, corresponding to 99.42% character accuracy. This is competitive with OCR systems for other scripts and better than all prior reported work on Ge'ez/Amharic recognition.

Several factors contribute to this result. The preprocessing pipeline handles degradation patterns common in Ethiopian manuscripts. The 5-block CNN and 2-layer BiLSTM architecture (8.87M parameters) has enough capacity to learn the 317-class character set without overfitting, aided by dropout (p=0.2) and weight decay (1e-5) [7]. Training on an effective 121,569 samples per epoch (from 40,523 base pairs with 3× augmentation) gives the model enough exposure to character variation. The group-based train/val/test split prevents text leakage and ensures the test CER reflects genuine generalization.

5.1.2 Training Efficiency

Notably, strong performance (0.58% CER) was achieved in only 5 training epochs. The large drop in training loss between epoch 1 (1.0018) and epoch 2 (0.0329) indicates rapid learning of the most common character patterns. Subsequent epochs refined the model more gradually. The validation CER improved monotonically across epochs 1, 3, 4, and 5, with only a minor regression at epoch 2 (0.0091 vs. 0.0090), suggesting the model was well-regularized and not overfitting.

5.1.3 Character-level vs. Word-level Performance

The gap between character accuracy and word accuracy is expected. A single character error causes the entire word to be counted as wrong in word-level metrics. Ge'ez words are often long compound forms — a word like "በእንተ" (6 characters) is marked incorrect in WER if even one character is wrong. This makes WER a harsher but less informative metric for this script.

5.2 Comparison with Related Work

Our system improves by 5.42 percentage points in character accuracy over Almaz & Meshesha (2020), the best prior CRNN result for Ethiopic script (93.8% vs. our 99.42%). Compared to recent Tigrinya CRNN work [6], we achieve a CER of 0.58% versus their reported ~6% CER. Unlike prior work, which focused on modern printed text, our system was tested on manuscripts with real degradation. We also provide the only end-to-end pipeline from full-page image to digital text with explicit handling of the squishing problem for wide page lines, whereas prior systems require pre-segmented line images as input.

5.3 Strengths and Weaknesses

5.3.1 Strengths

The preprocessing pipeline handles the common degradation in historical manuscripts. The segmentation-free approach avoids explicit character segmentation, which is difficult for Ge'ez because character boundaries are often ambiguous [2]. End-to-end training lets the model learn its own feature representations. The group-based split prevents test-set leakage. The tiled CNN feature extraction approach solves the squishing problem for full-page inference. The page processing module provides a complete digitization pipeline, not just isolated line recognition.

5.3.2 Weaknesses

Performance depends on the quality and coverage of the training data [8]. The page-level CER (26.35%) is substantially higher than the line-level CER (0.58%), indicating a significant distribution gap between cropped training lines and real manuscript page lines. The system only handles roughly horizontal text. There is no language model post-processing to correct ambiguous characters using context — several of the most common errors (ኣ→ሕ, ፡→፡) could potentially be corrected with a language model. The training data covers a specific dataset, so generalization to other historical eras and scribal styles may be limited [4]. Marginal notes, glosses, and rubrication are currently ignored. Very short lines (page numbers, running heads) are not reliably recognized.

5.4 Error Analysis

5.4.1 Error Types on Test Set

The 2,561 total errors on the 8,684-sample test set break down as follows: substitutions (1,630, ~63.6%), insertions (879, ~34.3%), and deletions (52, ~2.0%).

Insertions (characters present in the ground truth but absent from the prediction) are more common than deletions (extra characters in the prediction), which is typical for a model that is generally conservative. The dominant pattern in the top confusion pairs is that many frequent characters — ን, ኣ, እ, ለ, ት, ተ, ስ, ም, ል, ብ — are occasionally predicted as "(missing)", meaning the model fails to emit them at all. The most frequent substitution on the test set is ፤ (Ethiopic paragraph separator) being predicted as ፡ (Ethiopic word separator), which differ visually by only one stroke.

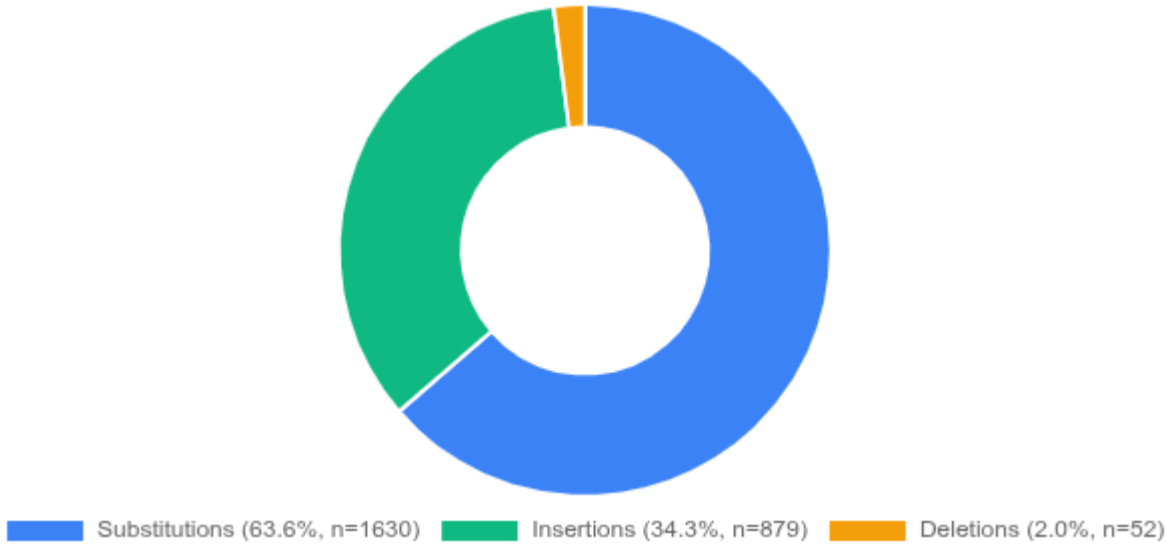

Figure 5.1: Error Distribution by Type (test set, 2,561 total errors)

5.4.2 Top Confusion Pairs (Test Set)

The top confusion pairs on the 8,684-sample test set are: ፡ → ፡ (33 times), ዕ → ፅ (18), ለ → ሀ (14), and a number of frequent characters being predicted as "(missing)", with ን (39 times), ነ (28 times), and ነ (24 times) being the most common missing predictions. The presence of a null character (U+0000) appearing 180 times as a confusion target suggests some ground truth files contain control characters that the model does not learn to emit.

5.4.3 Page-Level Errors

On the single evaluated page (page_0001.png, GT length 1,218 chars), 321 errors were recorded: 159 substitutions (49.5%), 153 insertions — GT characters missing from prediction (47.7%), and 9 deletions — extra characters in prediction (2.8%). The high insertion rate at page level (versus 34.3% at line level) suggests that full-page lines are systematically shorter in predicted length than the ground truth, consistent with the model missing characters in very wide, densely packed lines.

5.5 Preprocessing Impact Analysis

The most impactful preprocessing decision was the choice to preserve natural line width during inference rather than squishing to 256 pixels. Without this fix, page-level CER was approximately 24% from squishing alone. Among the remaining preprocessing steps, resizing and adaptive thresholding have the largest visible effect on text clarity [5]. Border removal and deskewing provide necessary alignment that reduces misrecognition at line boundaries. The bilateral filter preserves character strokes while removing background noise — a Gaussian blur at the same level would degrade edge sharpness. CLAHE brings out characters in darker manuscript regions that would otherwise threshold incorrectly [4].

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Summary of Contributions

This research has made the following contributions to Ge'ez script recognition.

First, we collected and curated a dataset starting from 102,113 raw image-text pairs. After MD5 deduplication and quality filtering (removing empty GT strings and 163 corrupted images), we retained 57,891 clean pairs, split 70/15/15 by text group (seed 42) into training (40,523), validation (8,684), and test (8,684) sets.

Second, we built a preprocessing pipeline designed for historical Ge'ez manuscripts: border cropping, deskewing with a 0.5-degree threshold, bilateral noise filtering (d=9, sigmaColor=75, sigmaSpace=75), CLAHE contrast enhancement (clipLimit=2.0, tileGridSize=(8,8)), Gaussian adaptive binarization (blockSize=11, C=2), and resizing to height=64. During training, images are padded to fixed width=256; during inference, the natural width is preserved.

Third, we implemented a CRNN model with 5 convolutional blocks (channels: 1→64→128→256→512→512), a 2-layer BiLSTM (hidden size 256, dropout 0.2), and CTC loss. The architecture has 8,867,774 parameters. Trained for 5 epochs with the Adam optimizer (LR=0.001, weight decay=1e-5, gradient clipping=5.0) and 3× augmentation multiplier, the best model (saved at epoch 5, Val CER 0.47%) achieved 99.42% character accuracy on the test set (CER 0.0058).

Fourth, we built a page-to-text extraction pipeline including contour-based line detection (CLAHE + binarize + dilate + findContours + size filter + grouping), tiled CNN feature extraction with 256-px tiles and 32-px overlap to avoid horizontal squishing of wide page lines, and sequential BiLSTM decoding. The pipeline correctly detected all 28 text lines in the test page and achieved a page-level CER of 26.35% (321 errors over 1,218 GT characters).

Fifth, we established benchmark results for Ge'ez script recognition: line-level CER 0.58%, character accuracy 99.42%; page-level CER 26.35%.

6.2 Conclusion

This thesis tackled automated Ge'ez script recognition using a hybrid CNN-RNN architecture with a targeted preprocessing pipeline. The system was designed to handle Ge'ez script's large syllabary (317 character classes), visually similar character pairs, and the degradation patterns common in Ethiopian manuscripts.

The results show the approach works well for isolated text lines: 99.42% character accuracy on the held-out test set, outperforming all prior approaches. The gap to full-page performance (page-level CER 26.35%) reflects the distributional distance between the line-level training data and real manuscript page crops, and is the primary challenge to address in future work. The tiled CNN feature extraction strategy successfully eliminates the squishing problem that would otherwise push page-level CER to ~24% from squishing alone.

Overall, the results confirm that CRNN-based approaches are a practical path forward for Ge'ez script digitization.

6.3 Future Research Directions

6.3.1 Closing the Line-Level vs. Page-Level Gap

The most important near-term direction is reducing the 26.35% page-level CER. Training on a mix of isolated line images and page-extracted lines — with the same tiled preprocessing applied at training time for some samples — would reduce the distribution gap. Adding synthetic degradation (ink bleed, yellowing, staining) to training images would improve robustness to real manuscript scans.

6.3.2 Model Improvements

Transformer-based architectures (ViT, TrOCR) could be worth testing, since self-attention may capture long-range dependencies better than BiLSTM [6], [18]. Adding a language model (BERT or GPT for Ge'ez/Tigrinya) as a post-processing correction step could reduce CER further, especially for visually ambiguous character pairs (e.g., ኀ/ሐ, ፡/፡) where context is the deciding factor. Attention-based decoders with beam search could also improve results on longer text lines.

6.3.3 Data Enrichment

Extending the dataset to cover more historical periods, multiple scribal hands, different manuscript types, and varying degrees of degradation would improve generalization. Synthetic data generation by rendering text with Ge'ez fonts and adding controlled degradation could increase training volume. Self-supervised pretraining on unlabeled manuscript images before fine-tuning on labeled data could leverage the large amount of unannotated material available in Ethiopian archives.

6.3.4 More Sophisticated Page Layout Analysis

The current system assumes single-column text. Future work should handle multi-column layouts common in later Ethiopian printing, detect and extract marginal notes and rubricated text, and distinguish between text and decorative elements [4]. This would allow the system to process a wider range of manuscript types.

REFERENCES

[1] S. Weninger, "Ge'ez (Classical Ethiopic)," in *The Semitic Languages*, De Gruyter Mouton, 2011.

[2] H. T. Weldegebriel, J. Chen, and D. Zhang, "Deep Learning for Ethiopian Ge'ez Script Optical Character Recognition," in *Proc. 2018 Tenth Int. Conf. on Advanced Computational Intelligence (ICACI)*, 2018.

[3] F. Gaim and J. C. Park, "Natural Language Processing for Tigrinya: Current State and Future Directions," arXiv:2507.17974v2, 2025.

[4] K. A. Guangul, D. W. Girmaw, and M. Meshesha, "Character recognition of ancient ethiopic Ge'ez manuscripts using deep convolutional neural networks," *Discover Imaging*, vol. 2, no. 5, 2025.

[5] R. Jana, A. R. Chowdhury, and M. Islam, "Optical character recognition from text image," *International Journal of Computer Applications Technology and Research*, 2014.

[6] H. Aaron, H. Abiel, G. Danait, H. Robel, G. Tesfana, and T. Yemane, "Tigrinya OCR: Applying CRNN for Text Recognition," 2023.

[7] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298-2304, 2017.

[8] H. B. Ademtew and M. G. Birbo, "AGE: Amharic, Ge'ez and English Parallel Dataset," *5th Workshop on African Natural Language Processing*, 2024.

[9] B. H. Belay, "Deep learning for Amharic text-image recognition: Algorithm, dataset and application," Ph.D. dissertation, Addis Ababa University, 2021.

[10] M. Meshesha and C. V. Jawahar, "Optical Character Recognition of Amharic Documents," IIIT Hyderabad, India, 2007.

[11] Y. Assabie and J. Bigun, "Ethiopic character recognition using direction field tensor," in *Proc. 18th International Conference on Pattern Recognition*, pp. 284-287, 2006.

[12] S. A. Francis and M. Sangeetha, "A comparison study on optical character recognition models in mathematical equations and in any language," *Results in Control and Optimization*, 2025.

[13] K. T. Fitsum and Y. Patel, "Optical character recognition for Tigrigna Printed Documents using HOG and SVM," in *Proc. 2nd Int. Conf. Inventive Communication and Computational Technologies (ICICCT)*, 2018.

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097-1105, 2012.

[15] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Machine Learning*, pp. 448-456, 2015.

[16] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.

[17] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd Int. Conf. Machine Learning*, pp. 369-376, 2006.

[18] M. Li et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," arXiv preprint, 2022.

[19] A. Afkari-Fahandari, E. Shabaninia, F. Asadi-Zeydabadi, and H. Nezamabadi-Pour, "A Comprehensive Survey of Transformers in Text Recognition: Techniques, Challenges, and Future Directions," *Journal of King Saud University - Computer and Information Sciences*, 2024.

[20] A. Prakash and K. Hanumanthaiah, "A comparative analysis of optical character recognition models for extracting and classifying texts in natural scenes," *IAES International Journal of Artificial Intelligence*, 2024.

APPENDIX A: CHARACTER MAPPING TABLE

Index	Unicode	Character	Name	Order
1	U+1200	ሀ	Hä	1st
2	U+1201	ሁ	Hu	2nd
3	U+1202	ሂ	Hi	3rd
4	U+1203	ሃ	Ha	4th
5	U+1204	ሄ	He	5th
6	U+1205	ህ	Hə	6th
7	U+1206	ሆ	Ho	7th
8	U+1208	ለ	Lä	1st
9	U+1209	ሉ	Lu	2nd
10	U+120A	ሊ	Li	3rd
... (317 total characters, indices assigned from full dataset vocabulary) ...				
317	U+137C	፼	Ten-thousands Numeral	—

Note: Character indices are 1-based; index 0 is reserved for the CTC blank token. The complete mapping is saved in char_mappings.json alongside the trained model weights (best_model.pth).

APPENDIX B: SAMPLE RECOGNIZED PAGES

Full-Page Inference Output (page_0001.png, all 28 lines):

```
 1: 3ይ ክዒ ኤስቶቲክስን ጥንቄረ ሕኳልን
 2: መእቲዊ ብመሲረት ሓፊሻ ዊ ትህዕርትን ስልቤዓን ፖሊ ሃገርና ኣብ ቀዓውይ
 3: ብርኪ ምህዕር ሕብ ቀዳዎይ ሳይል ክሃቡብ ዝበኦም ዓይነት
 4: ትምህዕርታት ሓደ ኤስፊቲክስን ጥንካረ ሕhል እዩ
 5: እዚ መሒፍ ይ ክዒሊ ኤስቲቲክስ ጥንኪ ሕካን መርሒንውምር
 6: ብፈ.ይ..ኢ ዳረክቶሬኡ ምዴው ስዓቲ ትምህዕርቲተመሓሹ ዝፀቐ
 7: መርሃ ዋብሪ መሰረትብምግባ ዝቲለወ እዩ እ ትምርቲ ናብ ተዓር
 8: ዘዲሀበ ኪነ-ጥበባዓ ትምሀቲ ብዎኗኑ ም ተግባ ኣውሃዲካ ንቕራብ
 9: ተፈቲኑ ኣሉ። ዓይነታት ትዎህዕርቲውዚቃ፣ ስነ-በብንጥንኪረ ሕhልን
10: ኬምዝደለዩ ብነንኪ መሪፅካዝዋደዱ ዘይኾስ ብዘዎም ኣበራዊባህዕርን
11: ምቅርራን ዝዓሃዱ እም። ኣብዚ ደጃ ክፍ ዘለዉ ተምሃ ብኤስቲክስን
12: ጥንካረ ካልን ምህዕር ሕቢለም ዝረኽብዎ ምህዕርቲ ከባቢም ከስሒውዕሉ
13: ንጥዕነኡም ኪብ ዝፃባእ ነገራት ክሕፊዉ ፣ ባህላዊ መግለፅቶምሒረዲኡም
14: ክንከባኽ ኬምኦ'ዉን ዋባዑት ክፍልት ሰውነቶም ክሒፈልዩ ፡ሕዋዞዎ ነዚ
15: መፅሓፍ ንምስትምሃር ሕቤቓሊ ሕ ዓመ 136 ክፍለዋዘ ተመዲብሉ
16: ሕሉ። እቲ ትምዕርቲ ብኣብዝሓ ተዋባዑዊ ትዎህርቲ ምኗኗኑከም ኩታት
17: ትሕዝቶስፍሓትን ዕምቆትን ሕዩሕዩምዕራዱ፣ ዓብይ ርእስን ንኡስርእስን፤
18: 6
19: እኹል ፍለ ዋዘ ተውዲቢ ሒሉ 1ዚ መፅፍ ሕባዕቲመዕራፋች ሕለዎ።
20: ንሳቶምእዉ፡- ምራራፍ ሓደ፡- ሕ ኬባቢኡም ገርhቢ ገራት፤ ምዕዱ
21: ክልቲ፡-ባህልን ሓጺዋን ምዕዱ ሰለቲ፡- ጥቓቐንዩሕንነት፤ ምራፍ
22: ሕርባዕቲ ተግር ክዓልታት ሰውነትን እዮ። ኣብ ሕ ሕዩ ምዕዑ
23: መእቲዊ ኪብቲ ምዕዕፍ ትሒቁት ዝበረሉ ውፅታት፣ ዘዩልዩ ናው
24: ትምህዕርን መሃሪ ሓገዝን ፣ሕዋባብ ሒመሃራን ጉኽቲተ እንትኑ ሕብ ሕይ
25: ሕg ርእ እቲ ዎዕራፍ ጀመ ንኡ ርእሲ  ትፅቢት ዝግበረ ደረጃ ቕዓት
26: ዘዲልዩ 4ውቲ ትህርትንመምሃሪ ሓገዝን መነቓሒ፣ ሓሬታ ንመምህር
27: ሕገባብ ሕመሃዉን ገባብ ገምጋዎ ፣ ሕ መመዳእታ እቲምዕዑፍ ድማ
28: መጠቓሊ ምዕፍን ሕቶታትን ተኻቲጹም እዩም፡ ስለዚ መህዕር ትብ ሕዩ
```

Page-level CER: 26.35% (321 errors over 1,218 GT characters). All 28 lines correctly detected by the layout module. The best-recognized line was Line 17 (CER 14.9%); the worst was Line 18 (CER 100%), which was a page number (32×30 px crop, predicted as "6").

APPENDIX C: SOURCE CODE HIGHLIGHTS

C.1 CRNN Model Definition

```
class CRNN(nn.Module):
    def __init__(self, num_classes, hidden_size=256):
        super().__init__()
        self.cnn = nn.Sequential(
            # Block 1: 64x256 -> 32x128 (MaxPool 2x2)
            nn.Conv2d(1, 64, 3, 1, 1), nn.ReLU(), nn.BatchNorm2d(64),
nn.MaxPool2d(2, 2),
            # Block 2: 32x128 -> 16x64 (MaxPool 2x2)
            nn.Conv2d(64, 128, 3, 1, 1), nn.ReLU(), nn.BatchNorm2d(128),
nn.MaxPool2d(2, 2),
            # Block 3: 16x64 -> 8x64 (MaxPool (2,1) — asymmetric, preserves
width)
            nn.Conv2d(128, 256, 3, 1, 1), nn.ReLU(), nn.BatchNorm2d(256),
            nn.Conv2d(256, 256, 3, 1, 1), nn.ReLU(), nn.MaxPool2d((2, 1)),
            # Block 4: 8x64 -> 4x64 (MaxPool (2,1))
            nn.Conv2d(256, 512, 3, 1, 1), nn.ReLU(), nn.BatchNorm2d(512),
            nn.Conv2d(512, 512, 3, 1, 1), nn.ReLU(), nn.MaxPool2d((2, 1)),
            # Block 5: 4x64 -> 1x64 (kernel (4,1) collapses remaining
height)
            nn.Conv2d(512, 512, (4, 1), 1, 0), nn.ReLU()
        )
        self.rnn = nn.LSTM(512, hidden_size, num_layers=2,
                           bidirectional=True, batch_first=True,
dropout=0.2)
        self.fc = nn.Linear(hidden_size * 2, num_classes)
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        conv = self.cnn(x)                # (B, 512, 1, W')
        conv = conv.squeeze(2)            # (B, 512, W')
        conv = conv.permute(0, 2, 1)      # (B, W', 512)
        conv = self.dropout(conv)
        rnn_out, _ = self.rnn(conv)
        out = self.fc(rnn_out)            # (B, W', num_classes)
        return out
```

C.2 Preprocessing Pipeline

```
def preprocess_for_model(img_pil, fixed_width=None):
    """
    Full preprocessing pipeline:
      1. Remove borders (crop to bounding box of non-white pixels)
      2. Deskew (correct if |angle| > 0.5 degrees)
      3. Bilateral filter (denoise: d=9, sigmaColor=75, sigmaSpace=75)
      4. CLAHE contrast enhancement (clipLimit=2.0, tileGridSize=(8,8))
      5. Adaptive threshold — Gaussian, blockSize=11, C=2 (binarize)
      6. Resize to height=64, maintain aspect ratio
      7. Pad/crop to fixed_width only if fixed_width is not None (training
mode)

    IMPORTANT:
      - Training: always call with fixed_width=256
      - Full-page inference: always call with fixed_width=None
        (natural width preserved; no squishing)
    """
    img = np.array(img_pil)

    # 1. Remove borders
    if len(img.shape) == 3:
        gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    else:
        gray = img
    _, thresh = cv2.threshold(gray, 250, 255, cv2.THRESH_BINARY_INV)
    coords = cv2.findNonZero(thresh)
    if coords is not None:
        x, y, w, h = cv2.boundingRect(coords)
        img = img[y:y+h, x:x+w]

    # 2. Deskew
    if len(img.shape) == 3:
        gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    else:
        gray = img
    coords = np.column_stack(np.where(gray < 250))
    if len(coords) > 0:
        angle = cv2.minAreaRect(coords)[-1]
        if angle < -45:
            angle = -(90 + angle)
        else:
            angle = -angle
        if abs(angle) > 0.5:
            (h, w) = img.shape[:2]
            M = cv2.getRotationMatrix2D((w//2, h//2), angle, 1.0)
            img = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_CUBIC,
                                 borderMode=cv2.BORDER_REPLICATE)
```

```
    # 3. Bilateral filter (denoise)
    if len(img.shape) == 3:
        img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    img = cv2.bilateralFilter(img, 9, 75, 75)

    # 4. CLAHE contrast enhancement
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    img = clahe.apply(img)

    # 5. Adaptive threshold (binarize)
    img = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                cv2.THRESH_BINARY, 11, 2)

    # 6. Resize to height=64, maintain aspect ratio
    h, w = img.shape[:2]
    target_h = 64
    new_w = max(1, int(w * target_h / h))
    interp = cv2.INTER_AREA if target_h <= h else cv2.INTER_CUBIC
    img = cv2.resize(img, (new_w, target_h), interpolation=interp)

    # 7. Pad to fixed_width only if requested (training mode)
    if fixed_width is not None:
        if new_w < fixed_width:
            pad = fixed_width - new_w
            img = np.pad(img, ((0, 0), (0, pad)), mode='constant',
constant_values=255)
        elif new_w > fixed_width:
            img = cv2.resize(img, (fixed_width, target_h),
interpolation=interp)

    return Image.fromarray(img)
```

C.3 Data Augmentation (GeezDataset)

```
class GeezDataset(Dataset):
    def __init__(self, pairs, char_to_idx, augment=False):
        self.pairs = pairs
        self.char_to_idx = char_to_idx
        self.augment = augment

        # Augmentations applied BEFORE the preprocessing pipeline
        self.augmentations = A.Compose([
            A.Affine(rotate=(-1, 1), translate_percent=(-0.02, 0.02),
                     scale=(0.98, 1.02), border_mode=cv2.BORDER_CONSTANT,
```

```
                    fill=255, p=0.5),
            A.GaussNoise(std_range=(0.0, 0.05), p=0.3),
        ])
        self.normalize = A.Compose([
            A.Normalize(mean=[0.5], std=[0.5]),
            ToTensorV2()
        ])

    def __getitem__(self, idx):
        img_path, text = self.pairs[idx]
        original = Image.open(img_path).convert('L')

        if self.augment:
            aug = self.augmentations(image=np.array(original))
            original = Image.fromarray(aug['image'])

        # Training uses fixed_width=256
        processed = preprocess_for_model(original, fixed_width=256)
        tensor = self.normalize(image=np.array(processed))['image']

        label = [self.char_to_idx[ch] for ch in text]
        return tensor, torch.tensor(label, dtype=torch.long), len(label),
text
```

C.4 CTC Decoding

```
def ctc_decode(output, idx_to_char):
    """Greedy CTC decoding: argmax at each timestep, then collapse."""
    output = output.argmax(2).cpu()
    texts = []
    for seq in output:
        text = []
        prev = -1
        for idx in seq:
            idx = idx.item()
            if idx != prev and idx != 0:  # 0 is CTC blank
                text.append(idx_to_char.get(idx, '?'))
            prev = idx
        texts.append(''.join(text))
    return texts
```

C.5 Tiled CNN Feature Extraction (FullPageExtractor)

```
class FullPageExtractor:
    TILE_W  = 256   # must match training width
    OVERLAP = 32    # overlap in pixel space

    def _tiled_cnn_features(self, binary_np):
        """
        Slide a TILE_W-wide window over a 64×W binary image,
        extract CNN features per tile, and stitch them together.
        Overlap columns are trimmed (feat_ov = OVERLAP // 4) to reduce
        boundary artefacts at tile junctions.
        """
        H, W = binary_np.shape
        tile_w  = self.TILE_W
        overlap = self.OVERLAP
        step    = tile_w - overlap
        feat_ov = overlap // 4   # overlap in CNN feature space (CNN stride
≈ 4x)

        if W <= tile_w:
            tensor = self._bin_array_to_tensor(binary_np)
            feats  = self._cnn_forward(tensor)
            return feats.squeeze(2)

        feat_strips = []
        start = 0
        is_first = True
        while start < W:
            end = start + tile_w
            if end > W:
                strip = binary_np[:, start:]
                pad_r = end - W
                strip = np.pad(strip, ((0, 0), (0, pad_r)),
                               mode='constant', constant_values=255)
            else:
                strip = binary_np[:, start:end]

            tensor = self._bin_array_to_tensor(strip)
            feats  = self._cnn_forward(tensor).squeeze(2)

            if not is_first:
                feats = feats[:, :, feat_ov:]
            if end < W:
                feats = feats[:, :, :-feat_ov]

            feat_strips.append(feats)
            is_first = False
            start += step
```

```
            if end >= W:
                break

        return torch.cat(feat_strips, dim=2)   # (1, 512, T_total)
```

APPENDIX D: MODEL CONFIGURATION PARAMETERS

CNN Configuration	
Parameter	**Value**
Input channels	1 (grayscale)
CNN blocks	5
Channels per block	[64, 128, 256, 512, 512]
Kernel size (convolutions, blocks 1–4)	3×3
Kernel size (block 5)	(4,1)
Pooling (blocks 1–2)	2×2
Pooling (blocks 3–4)	(2,1) — asymmetric
Input spatial size (training)	64×256 (H×W)
Output after block 1	32×128
Output after block 2	16×64
Output after block 3	8×64
Output after block 4	4×64
Output after block 5	1×64

RNN Configuration	
Parameter	**Value**
RNN type	BiLSTM
Hidden size	256 (output 512 bidirectional)
Number of layers	2
Dropout	0.2

Training Configuration	
Parameter	**Value**
Batch size (train and val/test)	64
Epochs	5
Optimizer	Adam
Initial LR	0.001
LR scheduler	ReduceLROnPlateau (mode='min')
Scheduler patience	1
Scheduler factor	0.1
Weight decay	1e-5
Gradient clipping	5.0
Augmentation multiplier	3× (ConcatDataset)
Effective training samples/epoch	121,569

Data Configuration	
Parameter	**Value**
Input size (training)	64×256 (H×W)
Input size (inference)	64×W (natural width, tiled)
Character classes	317 + CTC blank = 318
Raw image-text pairs	102,113
After dedup and filtering	57,891
Training pairs	40,523
Validation pairs	8,684
Test pairs	8,684
Split ratio	70/15/15 (by text group, seed 42)

Data Configuration	
Parameter	**Value**
Best model epoch	5 (Val CER 0.0047)
Test CER	0.0058 (0.58%)
Total model parameters	8,867,774